

RequestLoggingMiddleware

This document explains the **RequestLoggingMiddleware** implementation used in the FougeraClub ASP.NET Core application. The middleware is responsible for **conditional request logging**, **role-based exclusions**, and **multilingual log descriptions (Arabic & English)**.

Purpose

The middleware logs meaningful user actions into the database while avoiding noisy or unnecessary logs such as:

- Anonymous requests
- GET requests (unless explicitly allowed)
- Master role actions
- Error/status pages
- SignalR hub traffic

It also supports **Member vs Admin contexts**, **session-based user detection**, and **localized log messages**.

Namespace

```
FougeraClub.Middelware
```

Custom Attributes

NoLoggingAttribute

```
[AttributeUsage(AttributeTargets.Method | AttributeTargets.Class)]  
public class NoLoggingAttribute : Attribute { }
```

Usage: - Apply on controllers or actions to **completely skip logging**.

YesGetAttribute

```
public class YesGetAttribute : Attribute { }
```

Usage: - Allows logging for **GET requests**, which are normally ignored.

Middleware Class

RequestLoggingMiddleware

```
public class RequestLoggingMiddleware
```

Handles request interception, validation, filtering, localization, and persistence of logs.

Constructor Dependencies

```
RequestDelegate next  
ILogger<RequestLoggingMiddleware> logger  
HttpContextAccessor httpContextAccessor
```

Dependency	Purpose
RequestDelegate	Continue the HTTP pipeline
ILogger	Log internal middleware errors
HttpContextAccessor	Access session & context

InvokeAsync Method

```
public async Task InvokeAsync(HttpContext context, ApplicationDbContext db)
```

Main execution logic of the middleware.

Early Exit Conditions

The middleware **skips logging** in the following cases:

1. `[NoLogging]` attribute exists
2. No authenticated user
3. User role is **Master**
4. HTTP Status $\neq$ `200` or `302`

- 5. GET request without [YesGet]
 - 6. SignalR notification hubs
 - 7. Error / status pages
-

User Identification

Member Area

- User identified from **Session** (Email key)

Admin / Identity Area

- User identified from **Claims**
 - Role checked from database
 - Role Master → logging skipped
-

Route & Action Detection

The middleware extracts:

- Area
- Controller
- Action
- Route ID

Fallback logic exists to infer values from Request.Path when routing metadata is missing.

Edit vs Add Detection

```
if (routeId != null && actionName == "AddEdit")
```

- Converts AddEdit into logical Edit action for logging clarity.
-

Localization (Arabic & English)

Two cultures are used:

```
CultureInfo("ar")  
CultureInfo("en")
```

Log Message Structure

Arabic / English:

Performed an operation **[Action]** in page **[Controller]**

Generated dynamically using:

- Resource1
- Resource2

With fallbacks for singular/plural keys.

Special Cases Handling

Login / Logout Scenarios

Handled via **Session flags**:

- ShowToastrLoginSuccessfullyLoggedIn
- ShowToastrLoginSuccessfullyLoggedIn_Admin

Ensures accurate logs like:

- Login
 - Logout
 - Update My Data
-

Error Page Filtering

The following actions are ignored:

- HttpStatusCodeHandler
- Error
- Any action containing error

Prevents noisy system logs.

Database Persistence

```
db.RequestLogs.Add(log);  
await db.SaveChangesAsync();
```

Each valid request is saved with:

- Path
- Method
- Controller
- Action
- UserId
- Target (optional)
- Timestamp
- Arabic & English descriptions

Exception Handling

```
catch (Exception ex)
```

- Errors are logged via `ILogger`
- Request pipeline **continues safely**

Summary

This middleware provides:

- Smart request logging
- Role & area awareness
- Noise reduction
- Full localization
- High flexibility via attributes

It is designed for **enterprise-level audit logging** without harming performance or log quality.

 *Recommended for Admin-heavy systems requiring clean, meaningful activity tracking.*